Detecting Cyber Atrack in Web Application Using Honeypot with Distributed Log Management

TABLE OF CONTENTS

LIST OF FIGURES

CHAPTER 1
INTRODUCTION

1.1 OVERVIEW

Todays world is increasingly relying on computer networks. The increase in the use of network resources is followed by a rising volume of security problems. New threats and vulnerabilities are discovered every day and affect users and companies at critical levels, from privacy issues to financial losses. We are seeing targeted cyber attacks on organizations grow progressively more sophisticated, more serious, and more extensive. In the mid-2000s, the "black hat" community evolved from adolescent hackers bent on mayhem to organized crime networks, fueling highly profitable identity theft schemes with massive loads of personal data harvested from corporate and government networks. More recently, changes in IT infrastructure and usage models, including mobility, cloud computing, and virtualization have dissolved traditional enterprise security perimeters, creating a "target-rich" environment for hackers. But perhaps the most significant new element in the threat landscape is the emergence of highly targeted. long-term. international espionage and sabotage campaigns by covert state actors.

These long-term, state-sponsored campaigns are sometimes known as Advanced Persistent Threats (APTs). While APTs do represent a real danger in today's world, it is important to understand how they figure within a larger context. Only by separating reality from hype and seeing how APTs relate to the broader field of targeted attack methods and techniques will organizations be able to safeguard their information and operations in the coming decade.

1.2 Advanced Persistent Threat (APT)

The Advanced Persistent Threat (APT) attacks are special kind of slow moving attacks that are designed to defeat security controls

using unique attack vectors and malware specifically developed for the target organization. Aim behind APT attacks is not to disrupt services but to steal valuable data and intellectual property. Therefore, timely detection of APT attack is very important. We believe that deception tools like honeypots can significantly increase the possibility of early detection of such sophisticated attacks. In this research effort, a framework is proposed in which Honeypot along with NIDS is used to actively alert the administrator and not leaving the detection of APT in the hands to administrator by correlating different network events. The proposed framework is also implemented to test effectiveness of the proposed technique.

1.2.1 APT Honeypots

While there are many solutions to detect APT, are not all 100% effective. With the honeypot are able to some extent combat APT attackers. In this section we will discuss this problem and propose practical solutions that would form part of a system to detect APT. The concept of the honeynet first began in 1999 when Lance Spitzner, founder of the Honeynet Project, published the paper "To Build a Honeypot": "A honeynet is a network of high interaction honeypots that simulates a production network and configured such that all activity is monitored,

Honeypot is an information system whose purpose is to attract potential attackers and record their activities. Honeypot is used to detect and analyse attacks on computer networks and systems. Honeypots servers are dedicated servers, workstations and the network collects information about attackers and intruders who attack systems**.**

CHAPTER 2

LITERATURE SURVEY

2.1 Creation and Management of Social Network Honeypots for Detecting Targeted Cyber Attacks

Authors : Abigail Paradise, Asaf Shabtai

Reconnaissance is the initial and essential phase of a successful advanced persistent threat (APT). In many cases, attackers collect information from social media, such as professional social networks. This information is used to select members that can be exploited to penetrate the organization. Detecting such reconnaissance activity is extremely hard because it is performed outside the organization premises. In this paper, we propose a framework for management of social network honeypots to aid in detection of APTs at the reconnaissance phase. We discuss the challenges that such a framework faces, describe its main components, and present a case study based on the **results of a field trial conducted with the cooperation of a large European organization.**

2.2 Characterizing Honeypot-Captured Cyber Attacks: Statistical Framework and Case Study

Authors : Zhenxin Zhan, Maochao Xu

Rigorously characterizing the statistical properties of cyber attacks is an important problem. In this paper, we propose the first statistical framework for rigorously analyzing honeypot-captured cyber attack data. The framework is built on the novel concept of stochastic cyber attack process, a new kind of mathematical objects for describing cyber attacks. To demonstrate use of the framework, we apply it to analyze a low-interaction honeypot dataset, while noting that the framework can be equally applied to analyze high-interaction honeypot data that contains richer information about the attacks. The case study finds, for the first time, that long-range dependence (LRD) is exhibited by honeypot-captured cyber attacks. The case study confirms that by exploiting the statistical properties (LRD in this case), it is feasible to predict cyber attacks (at least in terms of attack rate) with good accuracy. This kind of prediction capability would provide sufficient early-warning time for defenders to adjust their defense configurations or resource allocations. The idea of "gray-box" (rather than "black-box") prediction is central to the utility of the statistical framework

2.3 Firewall log analysis and dynamic rule re-ordering in firewall policy anomaly management framework

Authors : **K Lubna**, **Robin Cyiac**

Today, there are more many ways to communicate than there were just a few years ago and among them, internet plays a major role. Firewalls are essential for a secure network communication to ensure that only trusted packets are transferred between the private and public network. In firewall, security policy is implemented based on the rules defined by the network administrator; that decides which packets can be allowed to an organization's private network. Manual definition of rules often results in anomalies in the policy. Therefore, an effective anomaly detection and resolution approach is needed. After resolving these conflicts, the rules can be re-ordered dynamically that improves the

efficiency of the anomaly management framework. With firewall log analysis, frequently used rules can be set as primitive rules, to which more security can be added.

2.4 Research and design on distributed firewall based on LAN

Authors : Xiong Zeng-gang, Zhang Xue-min

Although the theory of distributed firewall was proposed shortly, but because of its advantages relative to traditional firewall, business user oriented characteristics, it could meet customer demand for higher security. This paper proposed a firewall program suitable for network security in accordance with small and medium enterprise network security situation, systematically analyzed the distributed firewall policy management and policy actuator functions, features and related technologies, to discuss the overall design and structure of the log server module.

2.5 Anomaly detection using negative security model in web application

Authors : M Auxilia, D Tamilselvan

Today's combat zone for both ethical and unethical hackers is the web. Rapid growth of web sites and web applications gives way to deliver complex business applications through the web. As the web dependency increases, so do the web hacking activities. Web applications are normally written in scripting languages like JavaScript, PHP embedded in HTML allowing connectivity to the databases, retrieving data and putting them in the WWW site. A web application is vulnerable to many kinds of threats and attacks. In order to detect known attacks, some set of attack rules and detections are needed. In this paper, a negative security model based on misuse of web applications is used. This negative security model provides a Web Application Firewall(WAF) engine with a rule set, to ensure critical protection across every web architecture. reach web applications. This paper has been tested with apache web server's log file. We have tested

CHAPTER 3
PROPOSED WORK

3.1 INTRODUTION

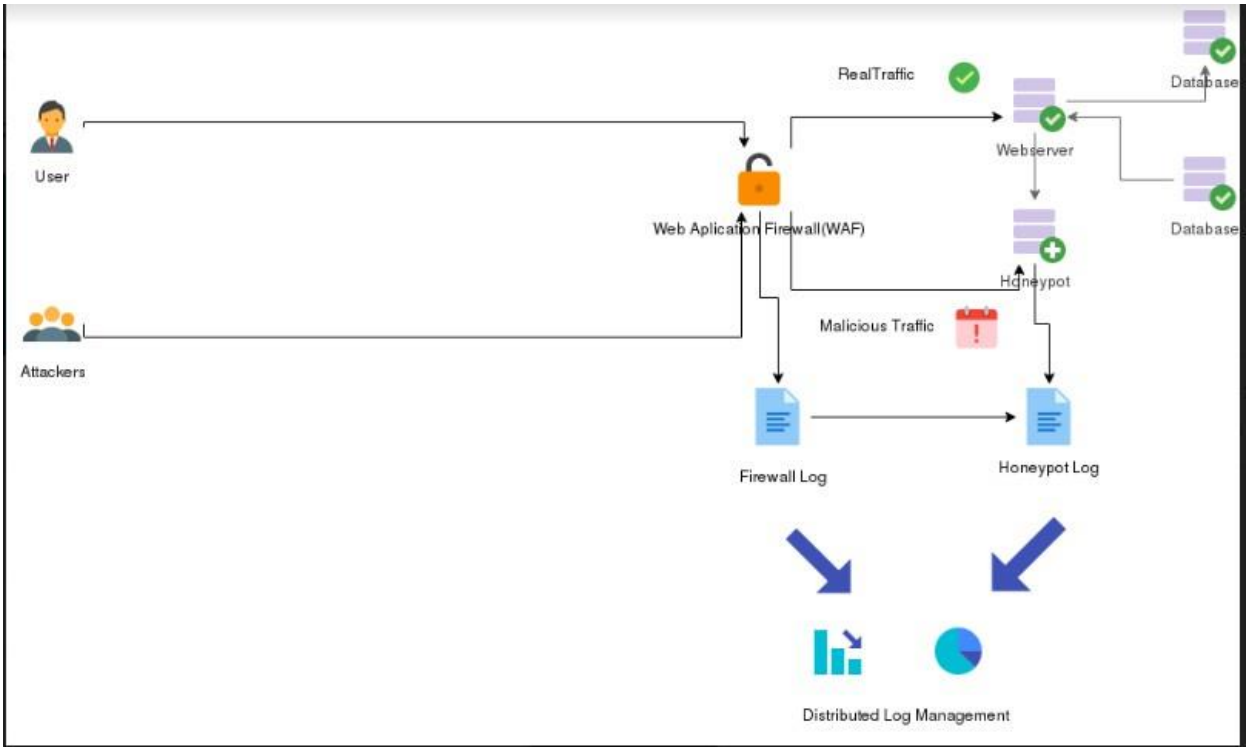
If the Attacker Performs any Malicious Scripts or Payload which is suspecious or harmfull to web application or Database.The Web application directly sends a Reverse Connection Backdoor to the Attacker at the Same Port no where the Attacker Connected to that site with When the backdoor installed Metasploitable framework capture the incoming connection from the attacker machine then we can track the activity of an attackerApart from this we analyse the honeypot log and fix these vulnerability in web applicationHere we perform two things one we perform a reverse attack to the attacker another we fixing the vulnerability in that webpageThis techniques can be useful to cyber defence to perfom reverse attack to the attacker and gain access to attacker machine.Also we develop a Log Management system to Keep track and alert these type of Activities in Web application

3.2 Architecture Diagram

The goal of our approach is the design and realization of a generic high interaction honeypot framework that allows to (semi-)automatically identify application layer based attacks (e.g. buffer overflows, format string attacks, etc.). Figure 1 depicts an example scenario that includes two attackers, a firewall / intrusion prevention system (IPS) and our honeypot framework. The purpose of the IPS / firewall is to filter the incoming traffic for known attacks. The honeypot framework itself consists of a proxy and a honeypot host. The proxy host is responsible for the session-individual logging of the network traffic that was sent to the honeypot. Furthermore, in case of a detected attack the proxy provides a mechanism to replay a specific previously logged session. An advantage

of the bipartite approach is that the honeypot and the log files are kept on separate hosts so that in case of a complete system takeover of the honeypot host by an attacker the log files remain save. Besides this, the replay mechanism that is integrated in the proxy can be used to analyze a discovered attack in detail, as well as to test if other system configurations are just as vulnerable as the honeypot service to the attack.

Fig 1: Architecture Diagram



3.3 Background

3.3.1 Network Security

Network malicious activity can be quantified and characterized through two distinct approaches: the first is to monitor production networks, where live hosts and devices are actually used by people; the second is to monitor an unused address space that nobody uses. The advantage of the second

approach over the first is that there is no user traffic to filter out. Indeed, the traffic received by unused addresses falls into three categories: malicious activity, misconfiguration, and backscatter from spoofed addresses. On the other hand, the disadvantage of the second approach is to rely on the assumption that malicious activity destined to unused addresses is similar to the one targeting production machines. Active tools include firewalls such as Netfilter, intrusion prevention systems (IPSs) such as Snort Inline, and vulnerability scanners such as Nessus. When monitoring an unused address space, passive tools are similar,but active tools are specific sensors developed with the only goal of better investigating the malicious activity received. Historically, unused address spaces were only passively monitored. Then researchers had the idea of actively replying to the traffic received to discover the exact threat behind each connection attempt. To understand the research challenges introduced with this new idea, we will now describe the different existing types of active sensors.

3.4. Honeypots

3.4.1. Definitions

We define a **network sensor** as an unused IP address instrumented to collect information about suspicious traffic. We separate sensors into two categories: passive sensors, which simply collect data without any interaction with the source of traffic; and active sensors, which can interact with the source of traffic to collect additional information. _ We define a honeypot as a network device that provides a mechanism for completing network connections not normally provided on a system and **logging** those connection attempts. We note that honeypot and active network sensor are synonyms. _ We define **a darknet** as a network of passive sensors. _ Similarly, we define a **honeynet** as a **network of honeypots**. _ By honeypot architecture, we mean a specific combination

of software solutions to administrate a **honeynet.** _ Finally by honeypot framework, we mean the combination of a honeypot architecture and a data processing solution to analyze malicious network activity.

3.5 Honeypots and Network Attack Processes

1. The first phase is to reach a victim, which means to send a communication attempt to a specific service hosted on a network device. For example, a technique for an attacker to discover a large number of victims is to scan incrementally all network addresses within a specific subnet. The attacker goes to the next phase only if the victim replies and the service is open to the attacker.

2. The second phase is to exploit the service found on the victim's machine by launching an attack payload. There is not always a clear boundary between the first and second phase, because some attacks are made of a single network packet , so communication attempts and attack payload overlap. Moreover, attackers often use the connection initialized during the scan to send the attack payload. The attacker goes to the next phase only if the service is successfully compromised by the attack launched.

3. The third phase is to use the newly corrupted victim's machine. The attacker can be someone who wants to gain access to a specific resource, or a worm that is simply spreading from one vulnerable machine to another. In such case, the worm installed on the newly corrupted machine will start probing for other victims and will create a new attack process starting with phase one again.

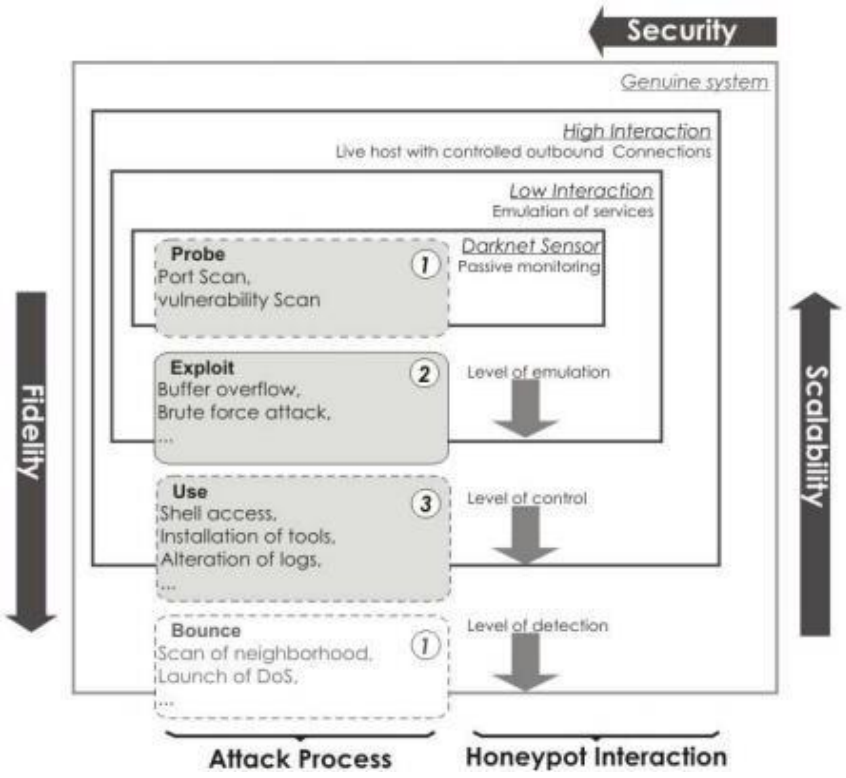


Fig 2: Network architecture

3.6 Intrusion Detection Systems with Snort

security is a big issue for all networks in today's enterprise environment. Hackers and intruders have made many successful attempts to bring down high-profile company networks and web services. Many methods have been developed to secure the network infrastructure and communication over the Internet, among them the use of firewalls, encryption, and virtual private networks. Intrusion detection is a relatively new addition to such techniques. Intrusion detection methods started appearing in the last few years. Using intrusion detection methods, you can collect and use information from known types of attacks and find out if someone is trying to attack your network or particular hosts. The information collected this way can be used to harden your network security, as well as for legal purposes. Both commercial and open source products are now available for this purpose.

Many vulnerability assessment tools are also available in the market that can be used to assess different types of security holes present in your network. A comprehensive security system consists of multiple tools,

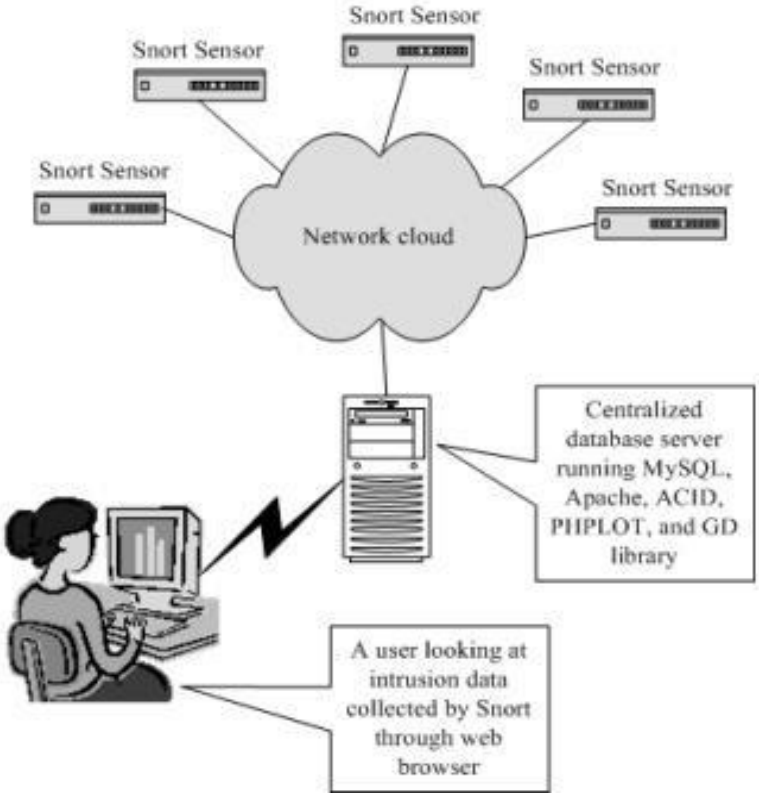


Fig 3 :**Multiple Snort sensors in the enterprise logging to a centralized database server.**

3.7 Some Definitions

3.7.1 IDS

Intrusion Detection System or IDS is software, hardware or combination of both used to detect intruder activity. Snort is an open source IDS available to the general public. An IDS may have different capabilities depending upon how complex and sophisticated the components are. IDS appliances that are a combination of hardware and

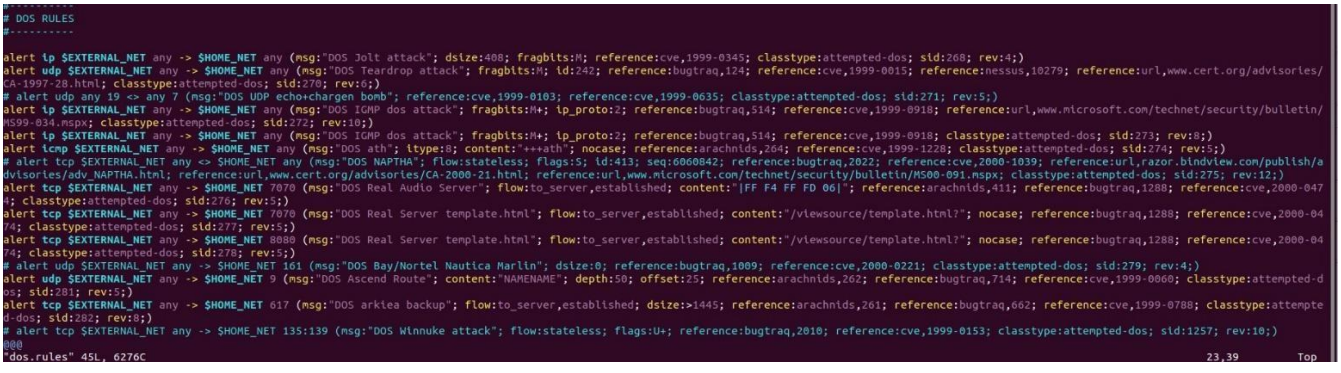
software are available from many companies. As mentioned earlier, an IDS may use signatures, anomaly-based techniques or both.

3.7.2 Network IDS or NIDS

NIDS are intrusion detection systems that capture data packets traveling on the network media (cables, wireless) and match them to a database of signatures. Depending upon whether a packet is matched with an intruder signature, an alert is generated or the packet is logged to a file or database. One major use of Snort is as a NIDS.

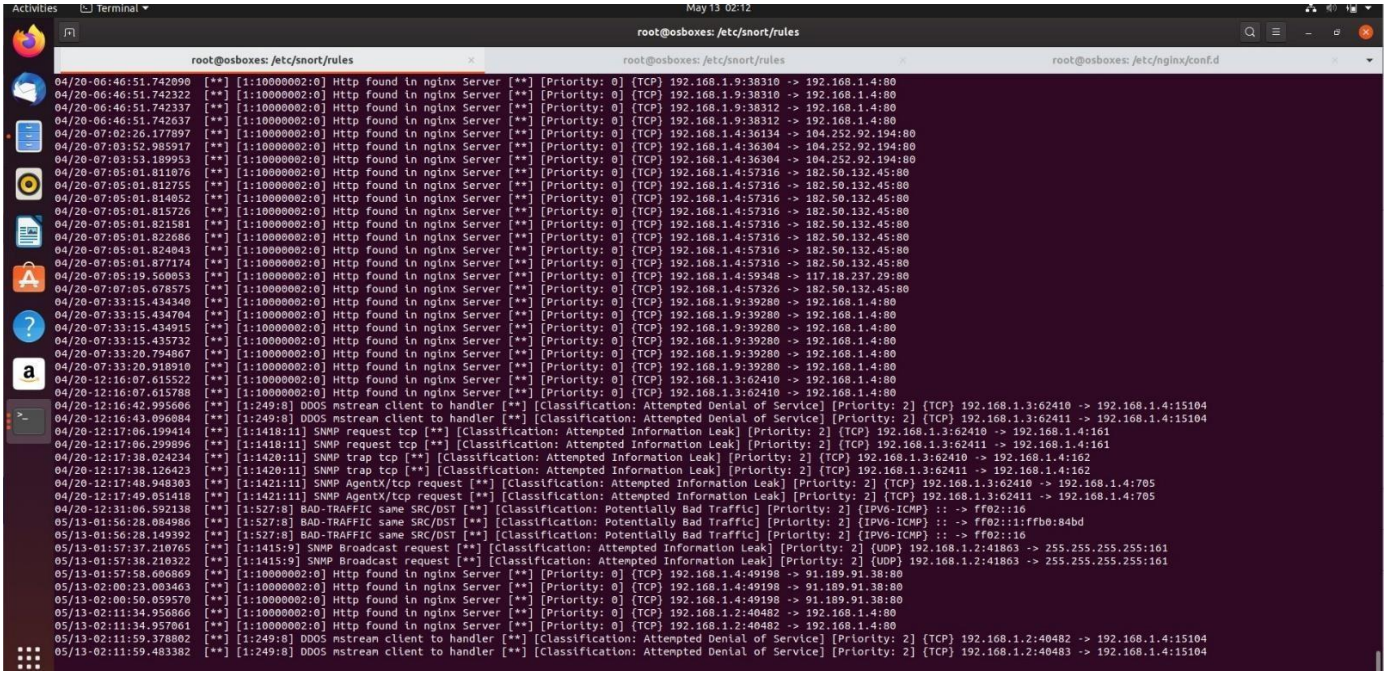
3.7.3 Alerts

Alerts are any sort of user notification of an intruder activity. When an IDS detects an intruder, it has to inform security administrator about this using alerts. Alerts may be in the form of pop-up windows, logging to a console, sending e-mail and so on. Alerts are also stored in log files or databases where they can be viewed later on by security experts. You will find detailed information about alerts later in this book. Snort can generate alerts in many forms and are controlled by output plug-ins. Snort can also send the same alert to multiple destinations. For example, it is possible to log alerts into a database and generate SNMP traps simultaneously. Some plug-ins can also modify firewall configuration so that offending hosts are blocked at the firewall or router level.



3.7.4 Logs

The log messages are usually saved in file. By default Snort saves these messages under /var/log/snort directory. However, the location of log messages can be changed using the command line switch when starting Snort. Log messages can be saved either in text or binary format. The binary files can be viewed later on using Snort or tcpdump program. A new tool called Barnyard is also available now to analyze binary log files generated by Snort. Logging in binary format is faster because it saves some formatting overhead. In high-speed Snort implementations, logging in binary mode is necessary.



In some cases you may need to disable some of the rules to avoid false alarms.

CHAPTER 4

IMPLEMENTATION

4.1 Implementation of a Real-Time Honeypot System for the Detection and Prevention of Systems Attacks

This chapter discusses the details of the network design used to investigate attacks on our honeypots and to detect and prevent this events in real time. The approach used for this network was a mixed one, using both literature studied and laboratory experiments.

4.1.1 Design of the Study

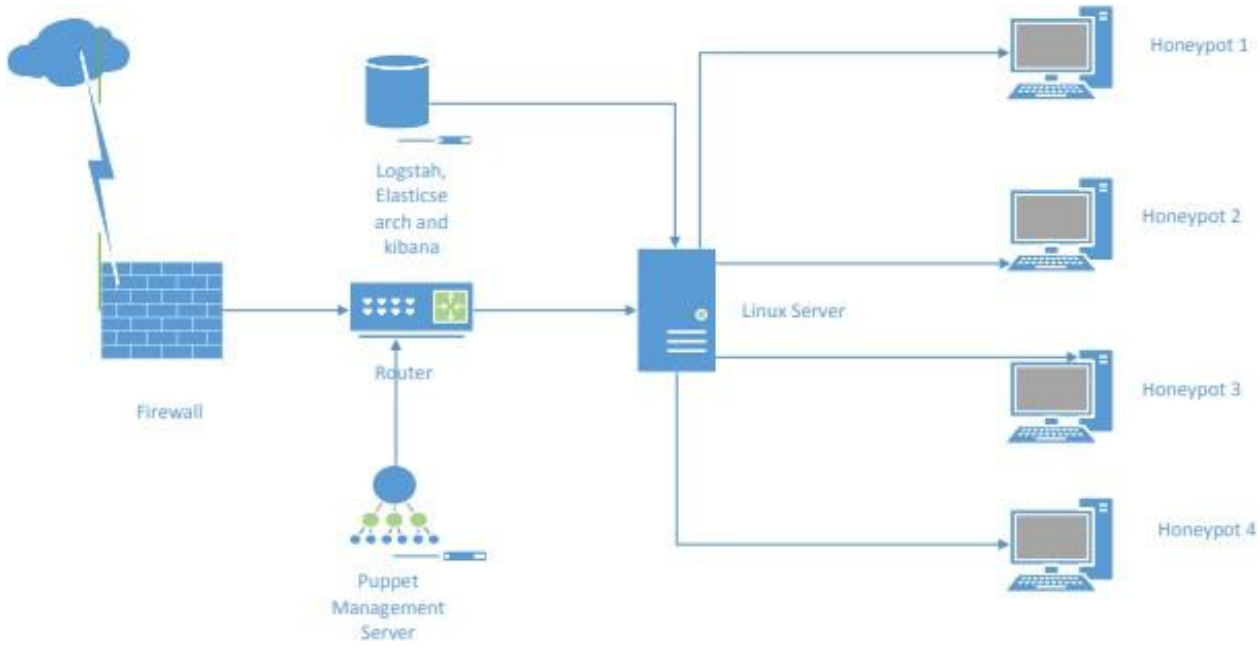


Figure 4: Network Design Diagram

4.1.2 Hardware and Software Environment

The honeynet system consist of Linux virtual machines running on VMware Workstation, version 12, hosted on Intel Core i7-4765T CPU, with 12 Gigabytes of RAM and 500 Gigabytes of storage. The virtual machine based honeynet has of 4 honeypot systems, a centralized logging host and Puppet automation host. Each honeypot system is consisted with three individual hosts; 21 a router, a firewall, and a Linux server host. The Linux server provided a Secure Shell (SSH) service to act as the target for attackers.

4.1.3 Data Collection

Data were collected using an open source data collection engine with real-time pipelining capabilities.. This was chosen because of its centralized log management which was used to collect all logs from compromised honeypot

4.1.4 Configure Honeypot In virtual Linux Machine

Log in to your Kali Linux machine as an admin user. Open a terminal window and download **pentbox** with the command:

```
wget
http://downloads.sourceforge.net/project/pentbox
18realised/pentbox-1.8.tar.gz
```

Once that file has finished downloading, extract the archive with the command:

```
tar xvfz pentbox-1.8.tar.gz
```

This will create a new directory named pentbox-1-8. Change into that new directory with *cd pentbox-1.8*.

The next step is to run the pentbox Ruby script with the command:

```
./pentbox.rb
```

Next, open a web browser on another machine (connect to the same network as the honeypot) and point it to http://SERVER_IP (where SERVER_IP is the IP address of your Kali machine). You should see an Access Denied error

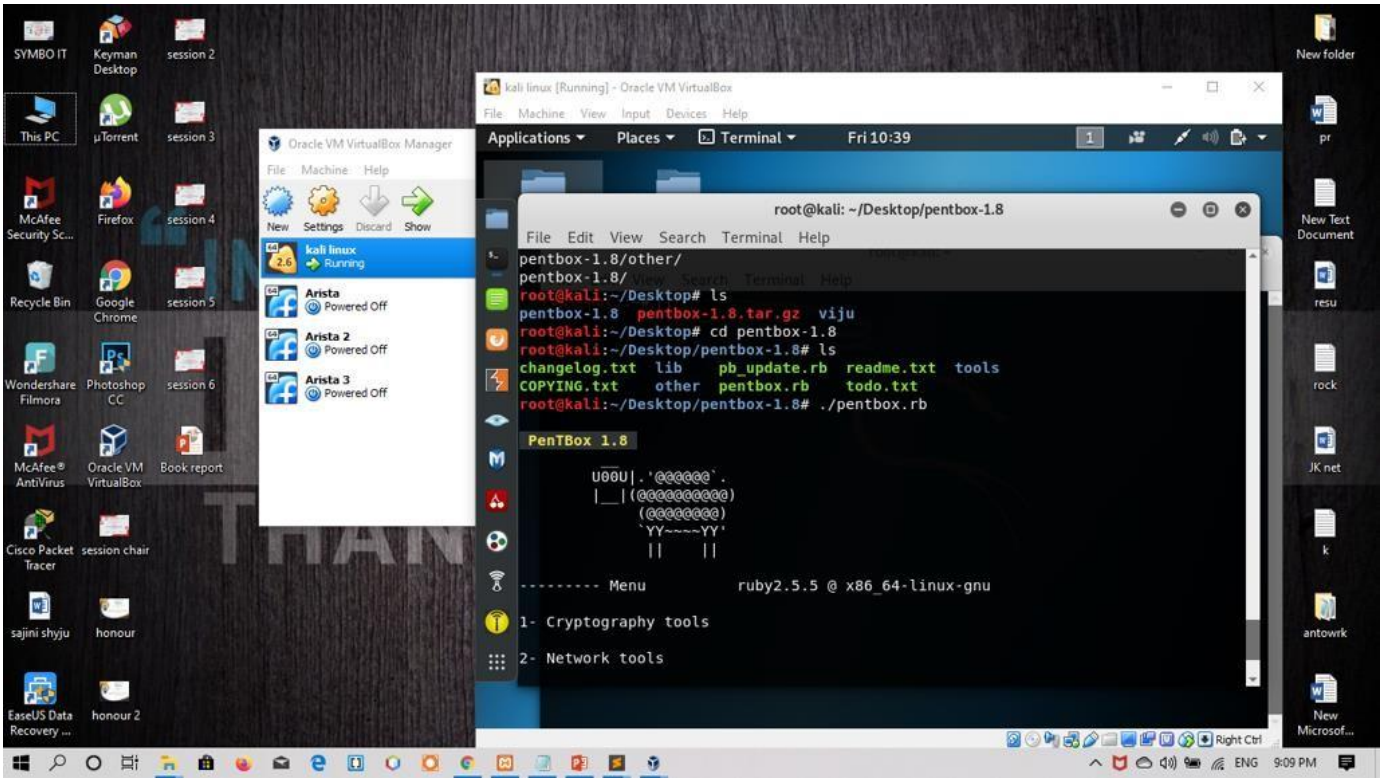


Fig 5: This will launch a honeypot listening on port 80

Next, open a web browser on another machine (connect to the same network as the honeypot) and point it to http://SERVER_IP (where SERVER_IP is the IP address of your Kali machine). You should see an Access Denied error

Go back to the terminal window on the Kali Linux machine and you should see the honeypot picked up the attempted connection

As you can see, pentbox logs the attempt, as well as the originating IP address.

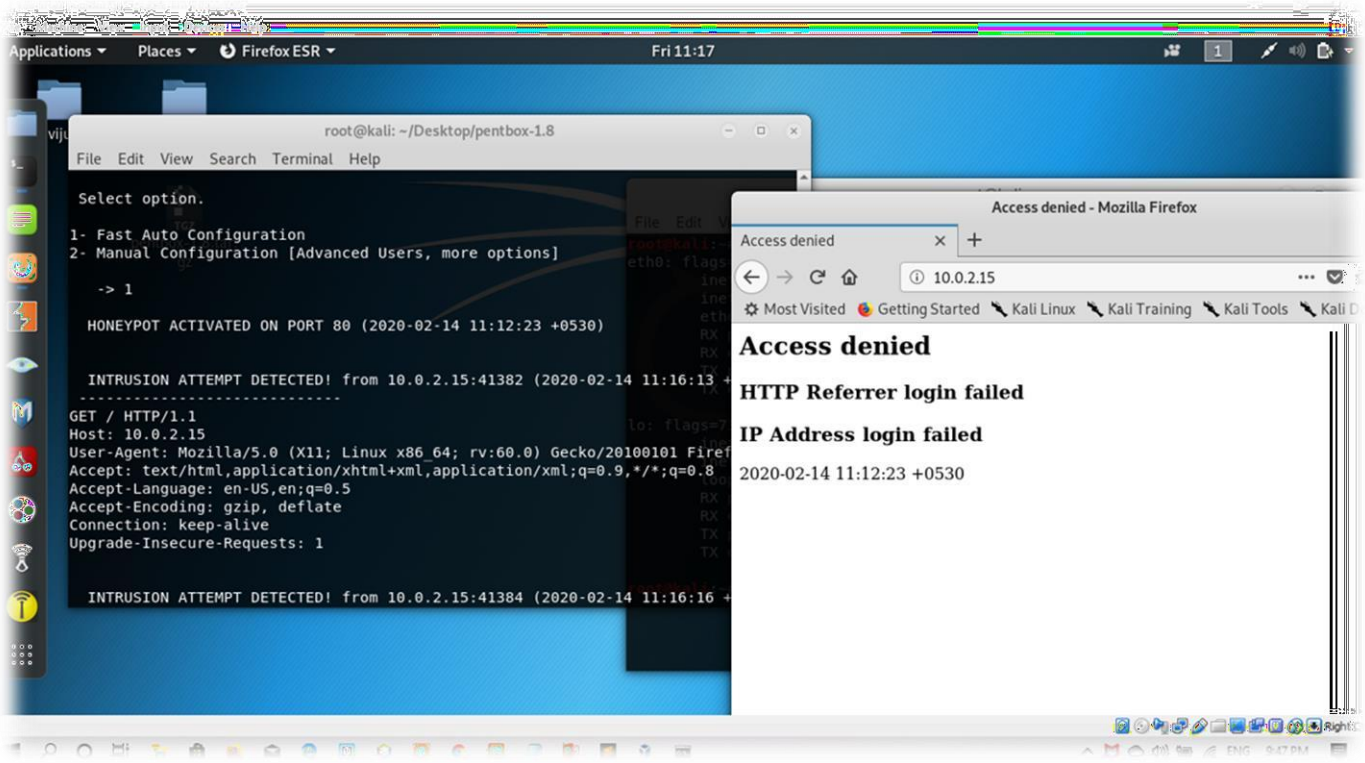


Fig 6: Attempt Detected

4.1.5 Install Nginx

Because we configured Kibana to listen on localhost, we must set up a reverse proxy to allow external access to it. We used Nginx for this purpose.

1. sudo apt-get install nginx apache2-utils
2. sudo htpasswd -c /etc/nginx/htpasswd.users kibanaadmin
3. sudo vi /etc/nginx/sites-available/default
4. See nginx config file
5. sudo service nginx restart

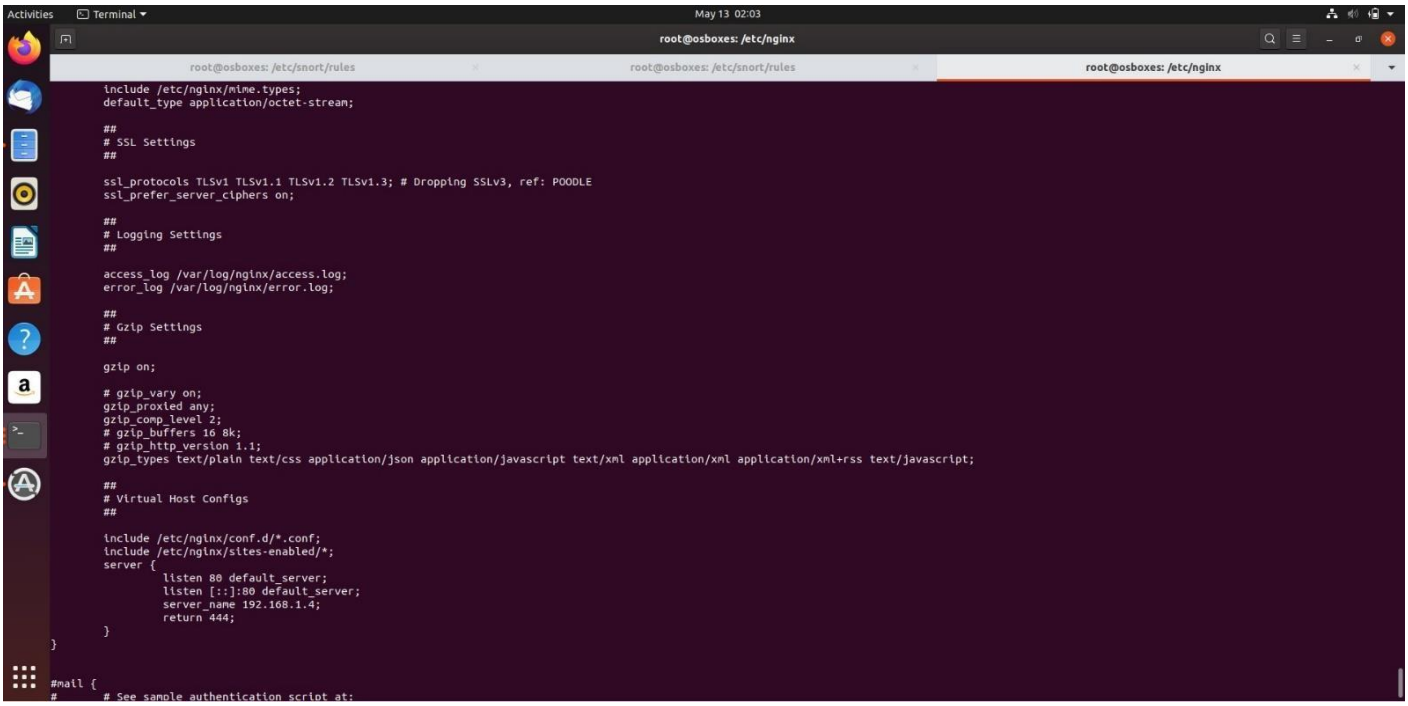


Fig 7 Nginx Configure

4.1.6 Installing Snort

In this section you will learn how to install precompiled version of Snort as well as how to compile and install it by yourself. Installation of the pre-compiled RPM package is very easy and requires only a few steps. However if you get Snort in source code format, the installation process may take some time and understanding. 2.2.1 Installing Snort from the RPM Package The installation procedure of Snort from the RPM package involves the following steps.

4.1.6.1 Download

Download the latest version from Snort web site (http://www.snort.org). At the time of writing this book, the latest binary file is snort-1.9.0-1snort.i386.rpm. 2.2.1.2 Install Run the following command to install Snort binaries: rpm --install snort-1.9.0-1snort.i386.rpm Installing Snort 29

This command will perform the following actions:

• Create a directory /etc/snort where all Snort rule files and configuration files are stored.

• Create a directory /var/log/snort where Snort log files will be stored.

• Create a directory /usr/share/doc/snort-1.9.0 and store Snort documentation files in that directory. You will see files like FAQ (Frequently Asked Questions), README and other files in this directory.

• Create a file snort-plain in /usr/sbin directory. This is the Snort daemon.

• Create a file /etc/rc.d/init.d/snortd file which is startup and shutdown script. On RedHat Linux, this is equivalent to /etc/init.d/snortd.

Basic installation is complete at this point and you can start using Snort. The version of Snort installed this way is not compiled with database support, so you can use it only for logging to files in the /var/log/snort directory. 2.2.1.3 Starting,

4.1.6.2 Stopping and Restarting Snort

To run Snort manually, use the following command:

/etc/init.d/snortd

start This command will start Snort and you can run the Snort daemon using the "ps –ef" command. You should see a line like the following in the output of this command:

root 15999 1 0 18:31 ? 00:00:01 /usr/sbin/

snort -A fast -b -l /var/log/snort -d -D -i eth0 -c /etc/

snort/snort.conf

Note that you have to start Snort manually each time you reboot the machine. You can automate this process by creating links to this file, which will be explained later in this chapter.

To stop Snort, use the following command:

```
/etc/init.d/snortd stop
```

To restart Snort, use this command:

```
/etc/init.d/snortd restart
```

4.2 Serve PHP with PHP-FPM and NGINX

4.2.1 Install and Configure PHP-FPM

Install the PHP process manager. On CentOS, Debian, and Ubuntu, the package name to install is php-fpm.

You can verify the PHP-FPM service is running with:

```
systemctl status php7.0-fpm.service
```

1. Depending on your distribution and PHP version, the PHP configuration files will be stored in different locations. This guide is using PHP 7.0 from Ubuntu's repositories on Ubuntu 16.04 as an example, and the `/etc/php/7.0/fpm/pool.d/www.conf` and `/etc/php/7.0/fpm/php.ini` files are what we'll be modifying.
   Find those full file paths using a `find` command:

```
find / \( -iname "php.ini" -o -name "www.conf" \)
```

   The output should look similar to:

```
root@localhost:~# find / \( -iname "php.ini" -o -name "www.conf" \)
/etc/php/7.0/fpm/php.ini
/etc/php/7.0/fpm/pool.d/www.conf
/etc/php/7.0/cli/php.ini
```

2. The `listen.owner` and `listen.group` variables are set to `www-data` by default, but they need to match the user and group NGINX is running as. If you installed NGINX using our *Getting Started with NGINX* series, then your setup will be using the `nginx` user and group. You can verify with:

3. `ps -aux | grep nginx`

   The output should be similar to:

```
root@localhost:~# ps -aux | grep nginx
root      3448  0.0  0.0  32500  3516 ?        Ss   18:21   0:00 nginx: master process /        usr/sbin/nginx -c /etc/nginx/nginx.conf
nginx     3603  0.0  0.0  32912  2560 ?        S    18:24   0:00 nginx: worker process
nginx     3604  0.0  0.0  32912  3212 ?        S    18:24   0:00 nginx: worker process
```

This shows the NGINX master process is running as root, and the worker processes are running as the nginx user and group. Change the listen variables to that:

```
sed -i 's/listen.owner = www-data/listen.owner = nginx/g' /etc/php/7.0/fpm/pool.d/www.conf
sed -i 's/listen.group = www-data/listen.group = nginx/g' /etc/php/7.0/fpm/pool.d/www.conf
```

4. When pairing NGINX with PHP-FPM, it's possible to return to NGINX a .php URI that does not actually exist within the site's directory structure. The PHP processor will process the URI, and execute the .php file, because its job is to process anything handed to it by NGINX. This presents a security problem.
   It's important to limit what NGINX passes to PHP-FPM so malicious scripts can't be injected into return streams to the server. Instead, the request is stopped, possibly then resulting in a 404. There are multiple ways to do this (see the NGINX wiki) but here we chose to specify the setting in PHP-FPM rather than in NGINX's configuration.

```
sed -i 's/;cgi.fix_pathinfo=1/cgi.fix_pathinfo=0/g' /etc/php/7.0/fpm/php.ini
```

   You'll notice that ;cgi.fix_pathinfo=1 is commented out by default. Setting it to 0 and uncommenting it will enforce the configuration should there be any upstream changes in the default value in the future.
5. Restart PHP-FPM to apply the changes:

```
systemctl restart php7.0-fpm.service
```

4.2.2 Configure the NGINX Server Block

we’ll start with a basic Server Block for a static HTTP page beingservedfrom /var/www/example.com.Replace example.com with your site’s domain or IP address, and the root directive with your site’s root directory.

/etc/nginx/conf.d/example.com.conf

```
server {
    listen          80 default_server;
    listen          [::]:80 default_server;
    server_name     example.com www.example.com;
    root            /var/www/example.com;
    index           index.html;

  location ~* \.php$ {
    fastcgi_pass unix:/run/php/php7.0-fpm.sock;
    include         fastcgi_params;
    fastcgi_param   SCRIPT_FILENAME    $document_root$fastcgi_script_name;
    fastcgi_param   SCRIPT_NAME        $fastcgi_script_name;
  }
}
```

Fig 8 Configure port

PHP Version 7.4.4

System	Windows NT DESKTOP-VLMV9Q8 10.0 build 18363 (Windows 10) AMD64
Build Date	Mar 17 2020 13:42:17
Compiler	Visual C++ 2017
Architecture	x64
Configure Command	cscript /nologo /e:jscript configure.js "--enable-snapshot-build" "--enable-debug-pack" "--with-pdo-oci=c:\php-snap-build\deps_aux\oracle\x64\instantclient_12_1\sdk,shared" "--with-oci8-12c=c:\php-snap-build\deps_aux\oracle\x64\instantclient_12_1\sdk,shared" "--enable-object-out-dir=../obj/" "--enable-com-dotnet=shared" "--without-analyzer" "--with-pgo"
Server API	Apache 2.0 Handler
Virtual Directory Support	enabled
Configuration File (php.ini) Path	C:\WINDOWS
Loaded Configuration File	C:\xamp\php\php.ini
Scan this dir for additional .ini files	(none)
Additional .ini files parsed	(none)
PHP API	20190902
PHP Extension	20190902
Zend Extension	320190902
Zend Extension Build	API320190902,TS,VC15
PHP Extension Build	API20190902,TS,VC15
Debug Build	no
Thread Safety	enabled
Thread API	Windows Threads
Zend Signal Handling	disabled
Zend Memory Manager	enabled
Zend Multibyte Support	provided by mbstring
IPv6 Support	enabled

Fig 9 Php-fpm configured

4.3 Rule Creation for Intrusion Detection

Snort supports a simple rule language that matches against network packets, generating alerts or log messages. Rules are broken into two logical areas: rule headers and rule options [8]. Rule headers contain required protocol fields that every rule must have and rule options contain a list of optional information used to refine a match.

The rule field format and an example rule are as follows:

alert tcp any any -> 192.168.1.0 21 (content:"USER")

The rule action tells Snort what to do when a match occurs. A common action, as shown in the example, is to log information to an alert file. The protocol field specifies one of four possible values: TCP, UDP, ICMP and IP. Each value has options specific to the protocol available for use in the option section. The source IP address field can contain the keyword "any", a single IP address or a CIDR (Classless Inter-Domain Routing) block. CIDR blocks allow for specifying ranges of IP addresses. Port numbers may be specified as a single static port, a range or use the keyword "any".

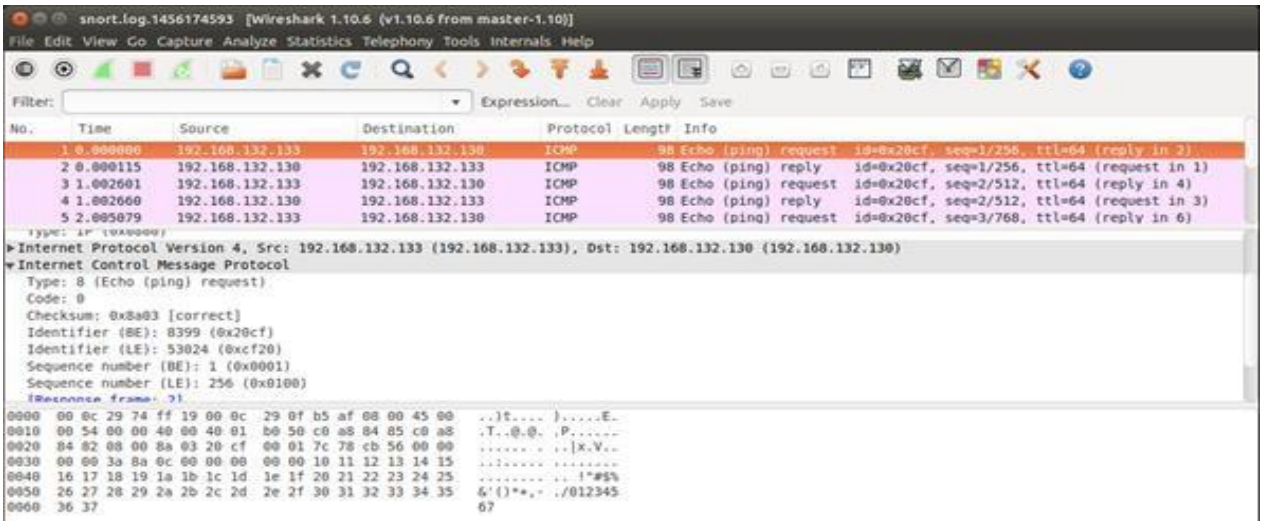
Field	Type	Range
proto	string	'icmp'
src	string or CIDR	'$SNET' or 0.0.0.0/0
sport	string or integer	'any'
dst	string or CIDR	'$DNET' or 0.0.0.0/0
dport	integer	'any'
itype	integer	0-255
icode	integer	0-255
icmp_id	integer	0-65535
icmp_seq	integer	0-65535
dsize	integer	0-65535
content	string	Any text

ICMP PACKET DETAILS.

S.no	Packet Details	Class Type
1.	icmp_id:667;itype:0;content:"ficken"	attempted-dos
2.	same as #1 except random IP's and IP identification field.	attempted-dos
3.	dsize:0;itype:8;icode:0	attempted-recon
4.	icode:0;itype:5	bad-unknown
5.	icode:0;itype:8;dsize:20; content:"abcde12345fghij6789"	trojan-activity
6.	itype:8;icode:0;dsize:32;content:"abcdefgh ijklmnopqr|0000|";depth:22;	trojan-activity
7.	icmp_id:123;icmp_seq:0;itype:0; content:"shell bound to port";	trojan-activity

4.3.1 Syntax for A Snort rule

```
     log tcp !192.168.0/24 any -> 192.168.0.33 (msg:
"mounted access" ; )
```



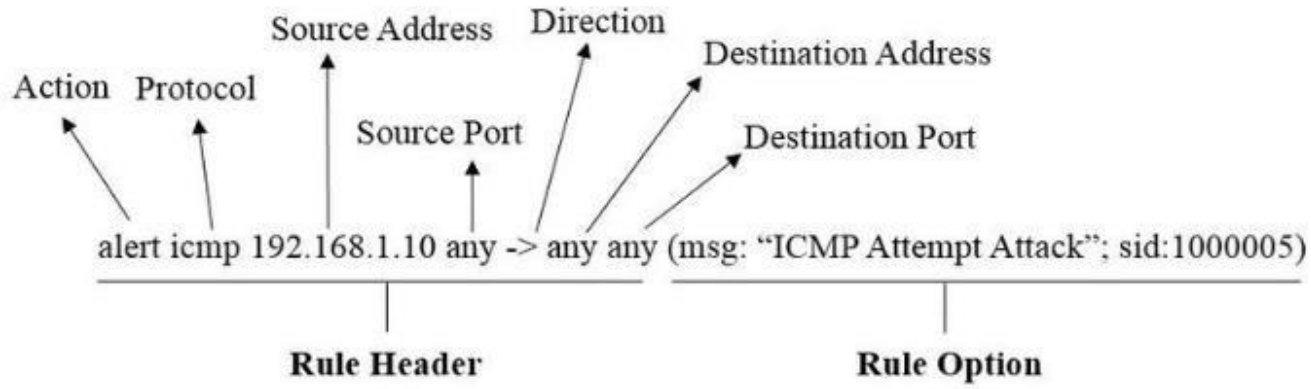


Fig 10 syntax of snort rule

Example of Snort IDS Rule. The rule options of Snort consist of two parts: a keyword and an argument (defined inside parentheses and separated by a semicolon). The keyword options are separated from the argument by a colon. A Snort rule can have one (or multiple) options within a rule option. Examples of keyword options are msg, ttl, tos, and icode.

4.3.2 Order of Rules Based upon Action

The five types of the rules can be categorized into three basic types.

1. Alert rules
2. Pass rules
3. Log rules

When a packet is received by Snort, it is checked in this order. Each packet has to go through all Alert rule checks before it is allowed to pass. This scheme is the most secure since no packet passes through without being checked against all alert types. However most of the packets are normal traffic and do not show any intruder activity. Testing all of the packets against all alert rules requires a lot of processing power. Snort provides a

way to change this testing order to one which is more efficient, but more dangerous.

Chapter: 5
RESULT AND DISCUSSIONS

5.1 Data Analysis and Results

The experiment mirrors real world implementation of honeynet system because a Webserver, File Transfer Protocol, and Simple Mail Tranfer Protocol and MYSQL servers were installed on the honeypots. The attack data was analyzed by calculating the frequencies, percentages, Chi square and the P-value for each of the honeypots. The data analysis investigated the top source IP addresses

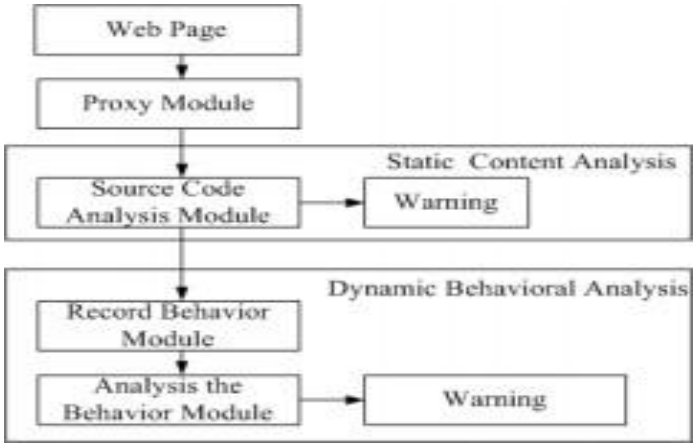


Fig 11 System architecture

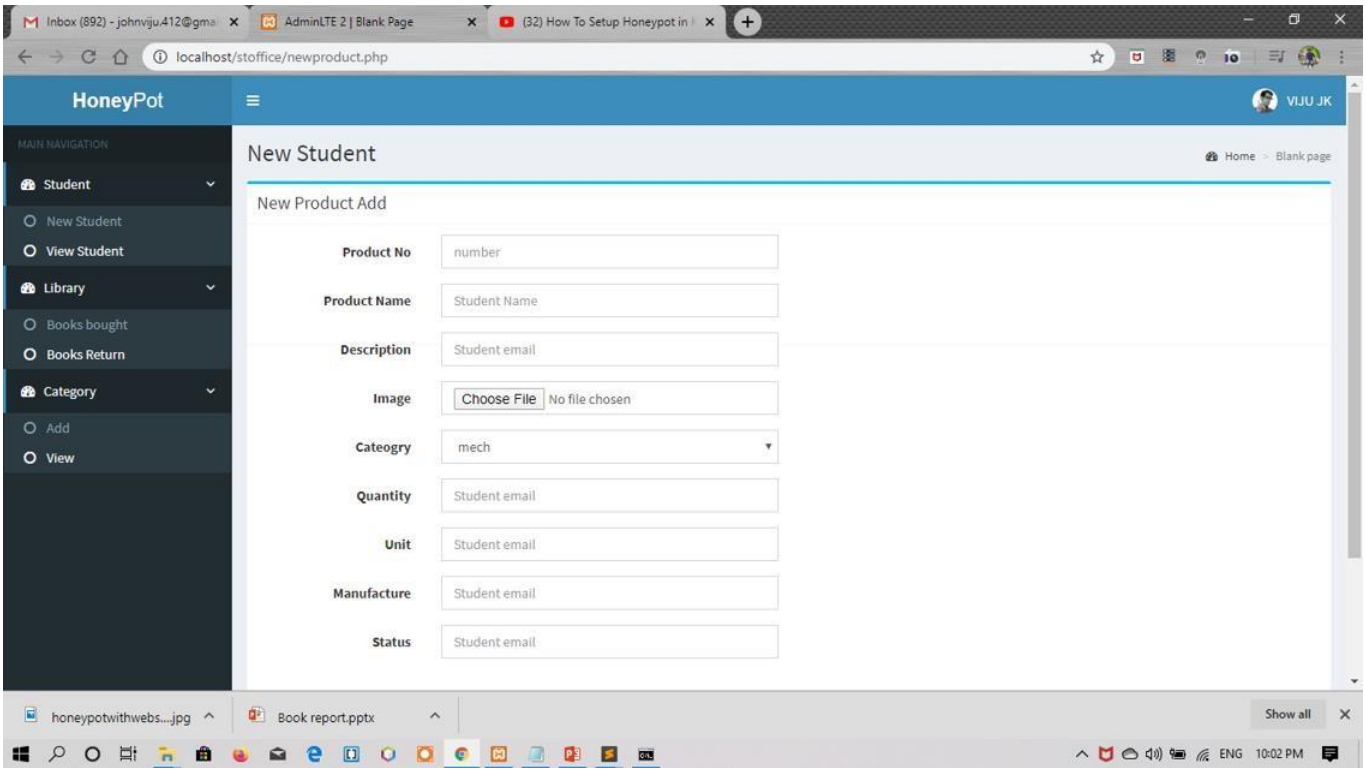


Fig 12 Website View

5.2 Log Data and Results

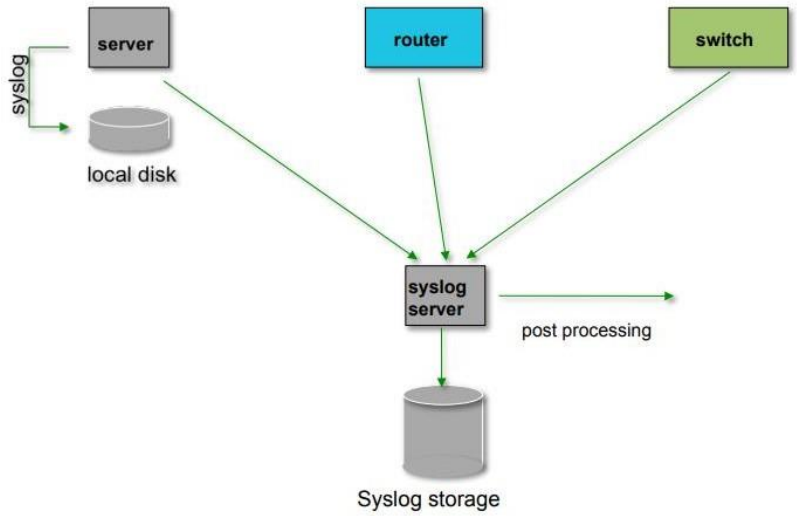


Fig 13 Centralized logging

5.3 Receiving syslog messages

Identify the facility that the equipment is going to use to send its messages

. • Reconfigure syslogd to listen to the network. - Ubuntu: add ”-r” to /etc/defaults/syslogd

• Add an entry to syslodg where messages are going to be written: local7.* /var/log/routers

• Create the file touch /var/log/routers

• Restart syslogd /etc/init.d/syslogd restart

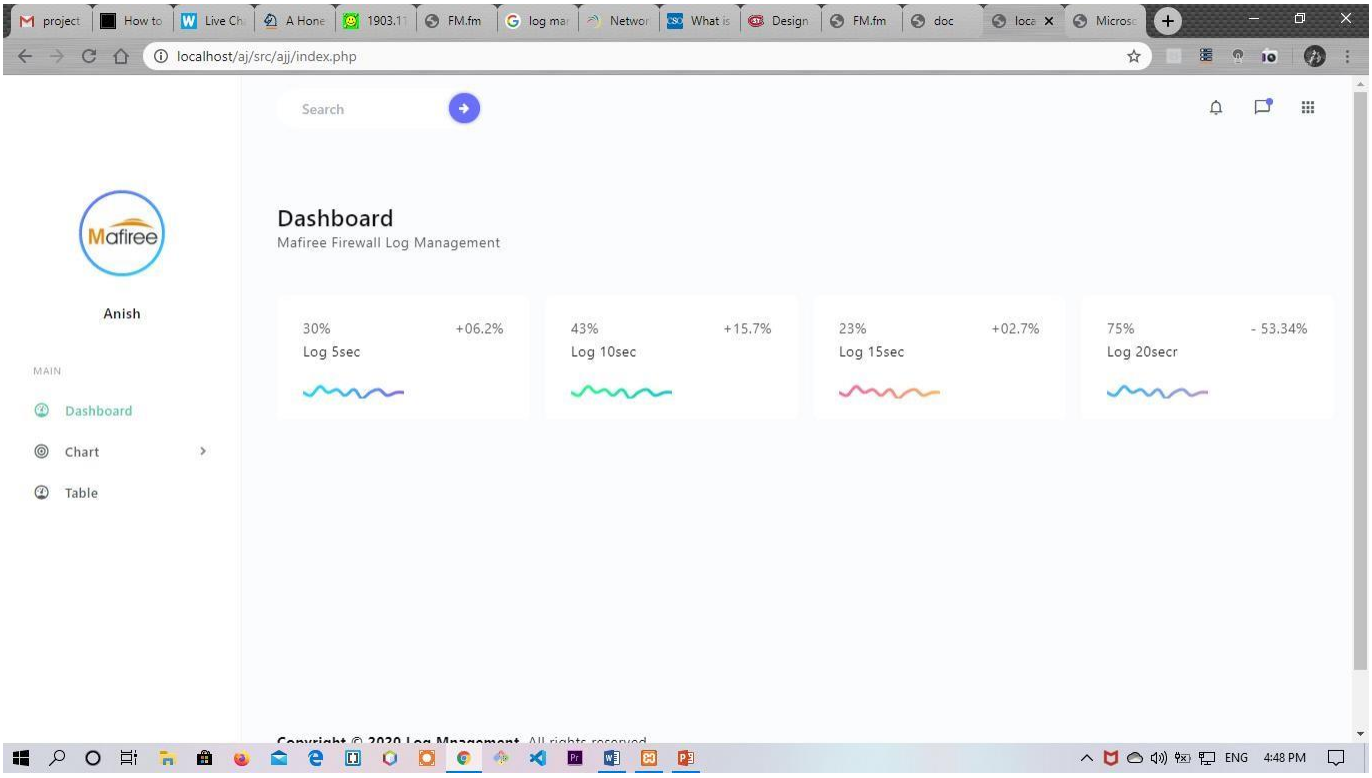


Fig 14 Log Management

5.4 Log in Chat View

5.4.1 Bar Chart

A **bar chart** or bar graph is a chart or graph that presents categorical data with rectangular bars with heights or lengths proportional to the values that they represent. The bars can be plotted vertically or horizontally. ... A bar graph shows comparisons among discrete categories.

And here we display all the incoming IPs in the port number as well as connection messages representation

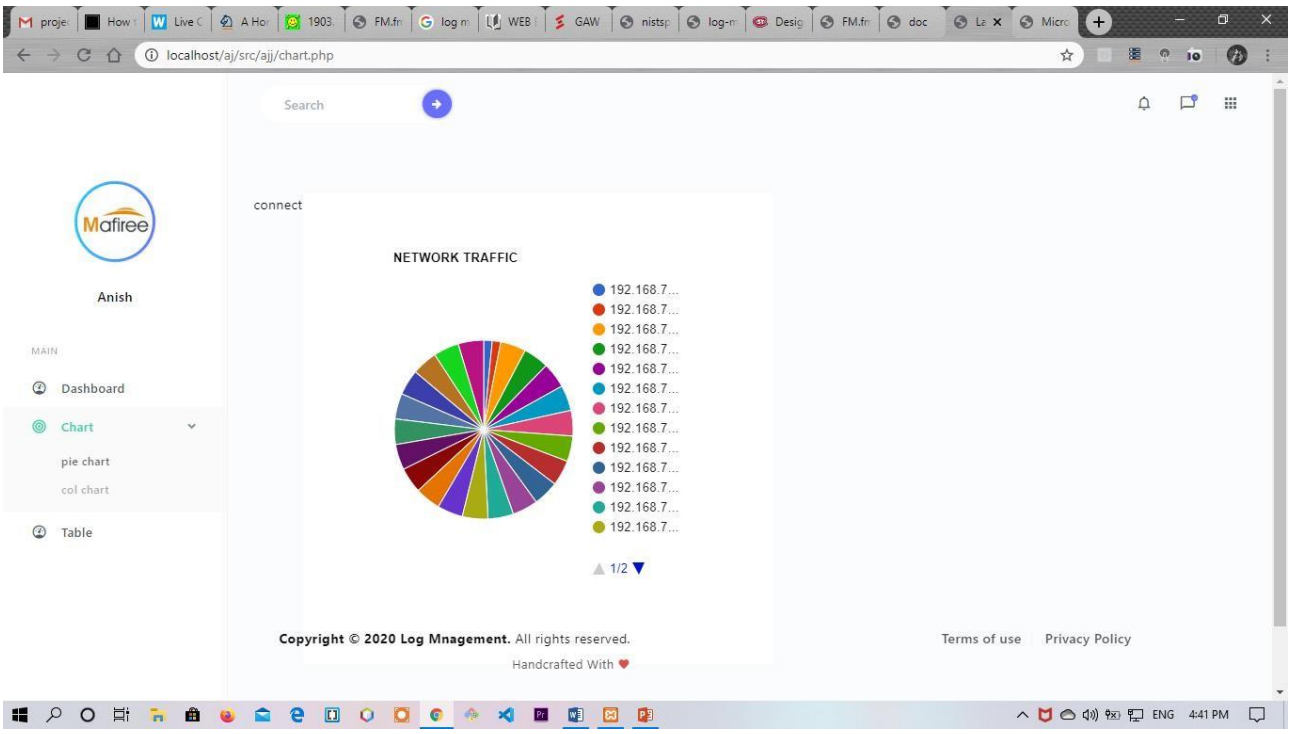


Fig 15 Bar Chart Log

5.4.2 Column Chart

A **column chart** is a graphic representation of data. Column charts display vertical bars going across the chart horizontally, with the values axis being displayed on the left side of the chart. ... As can be seen in this example, you can immediately see a gentle increase of users without reading any data

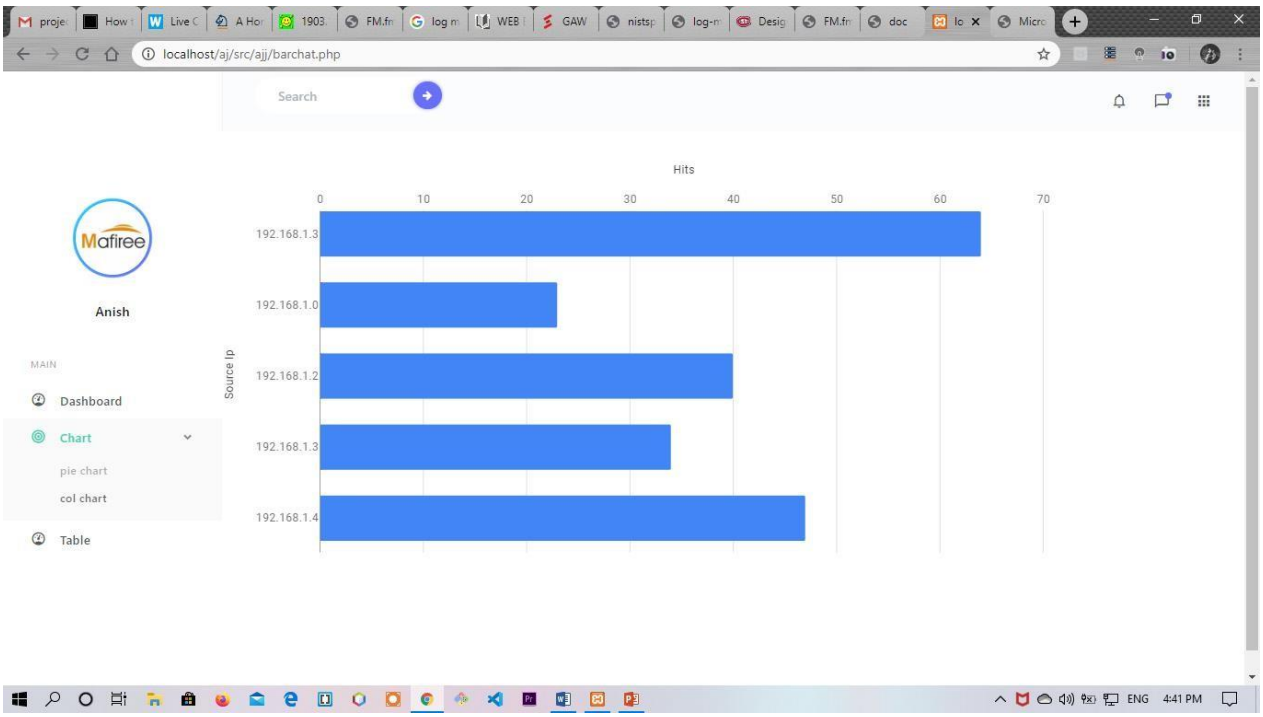


Fig 16 Column Chart

And also we display all the incoming IPs in the port number as well as no of hits representation

5.4.3 Log Table

And the noe the final result of all the logs are captured and stored database and they display in our website

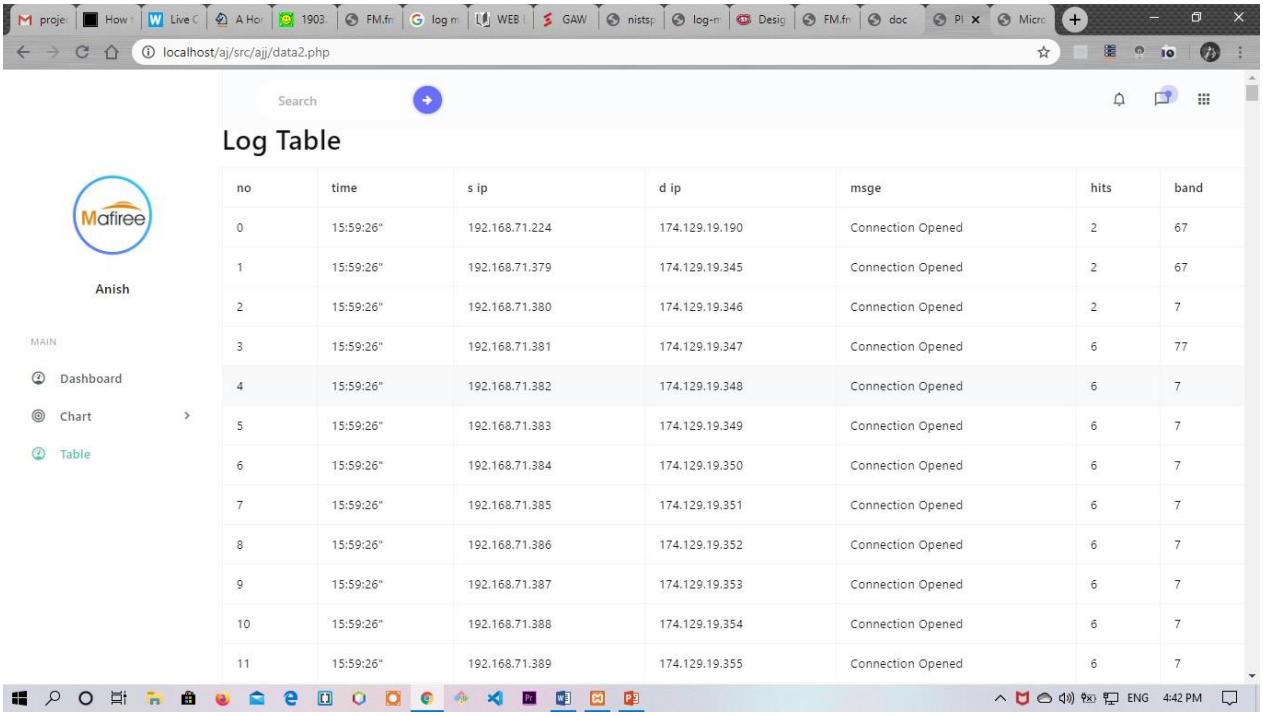


Fig 17 Log Table

Chat Bot Security

According to DZone, chatbot security risks come down to two categories – threats and vulnerabilities. Threats that a chatbot could pose include spoofing/impersonating someone else, tampering of data, and data theft. Vulnerabilities on the other hand, according to DZone, “are defined as ways that a system can be compromised that are not properly mitigated. A system can become vulnerable and open to attacks when it is not well maintained, has poor coding, lacks protection, or due to human errors.” Threats are often one-off events such as malware attacks or distributed denial of service (DDoS) attacks. Vulnerabilities are long term issues that need to be addressed regularly.

A Back Door Open To Hackers

Chatbots are nowadays mostly used in industries such as retail, banking, financial services, and travel that handles very crucial data such as credit/debit cards, SSN, bank accounts, and other Sensitive PII (Personally identifiable information).
The aggregation of such data is crucial for the chatbot to perform. Thus, it is required that chatbots are not vulnerable to be exploited by any hackers.
A recently released report from MIT Technology Review and Genesys showed that 90% of companies are already using AI strategies to increase revenue. The research also found that on average, between 25% and 50% of all customer queries can be solved through automated techniques. This has made it easier than before to handle complex tasks.

Mitigation

1. End-to-End Encryption

It is very important to have E2EE; this ensures that the whole conversation will be encrypted. Data in transit can also be tampered and spoofed; different protocols exist to provide encryption while addressing these issues — we will take a look at them later. Encryption, in some cases, might need to be used with authentication and integrity protection schemes.

As you may know, chatbots can also connect to channels like Facebook Messenger, Telegram, Slack, etc. In this case, the ideal mitigation for E2E is to only allow chatbots to connect to channels that support data encryption. Article 32 (a) of the General Data Protection Regulation (GDPR) specifically requires that companies take measures to pseudonymize and encrypt personal data.

The good news is that, thanks to regulations like GDPR, more companies are paying attention to data encryption and, therefore, giving

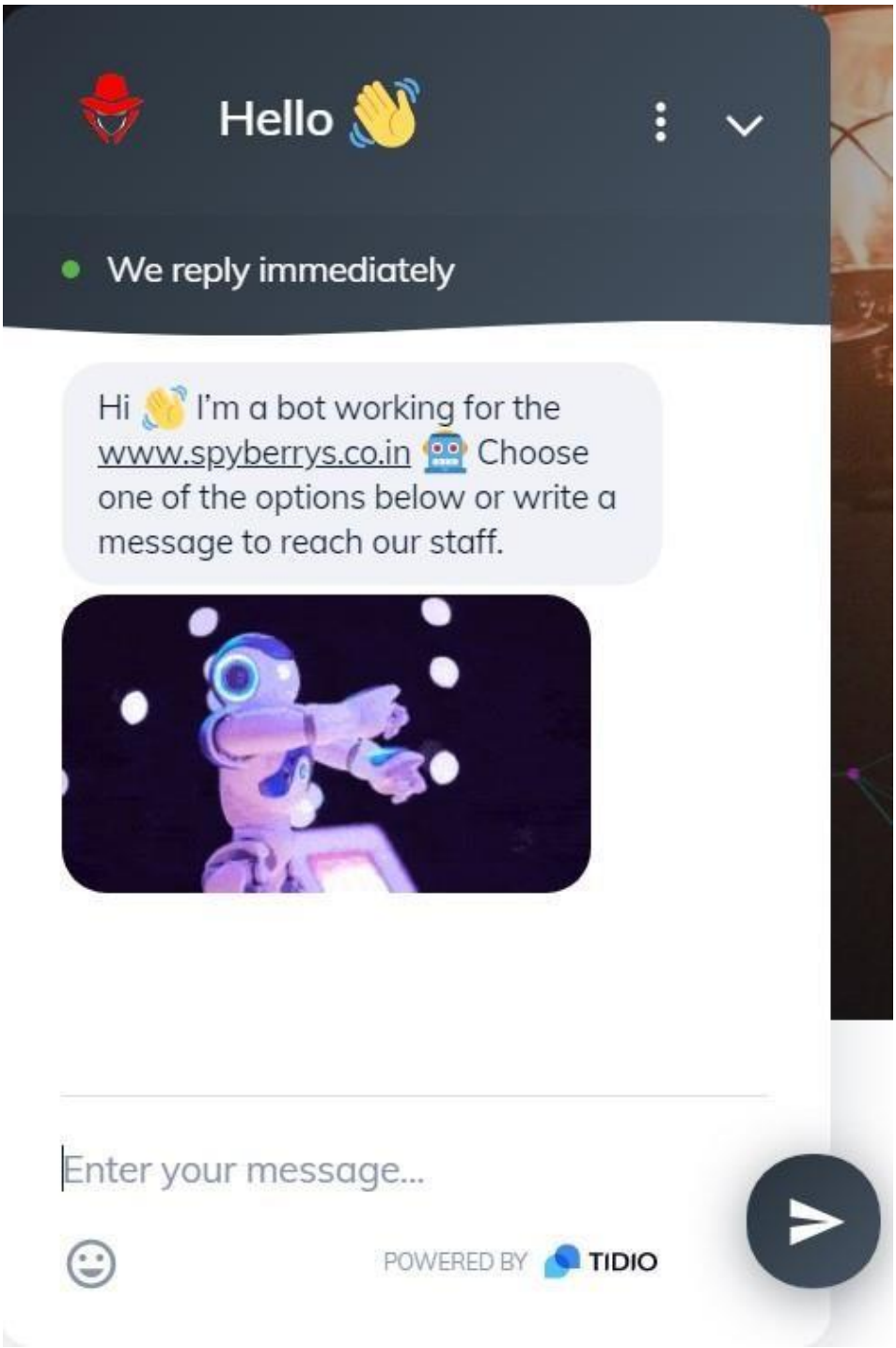
chatbots more channels to connect to in a secure manner. For example: in 2016, Facebook Messenger introduced the new feature called “Secret Conversations” that enables E2E based on Signal Protocol developed by Open Whisper Systems.

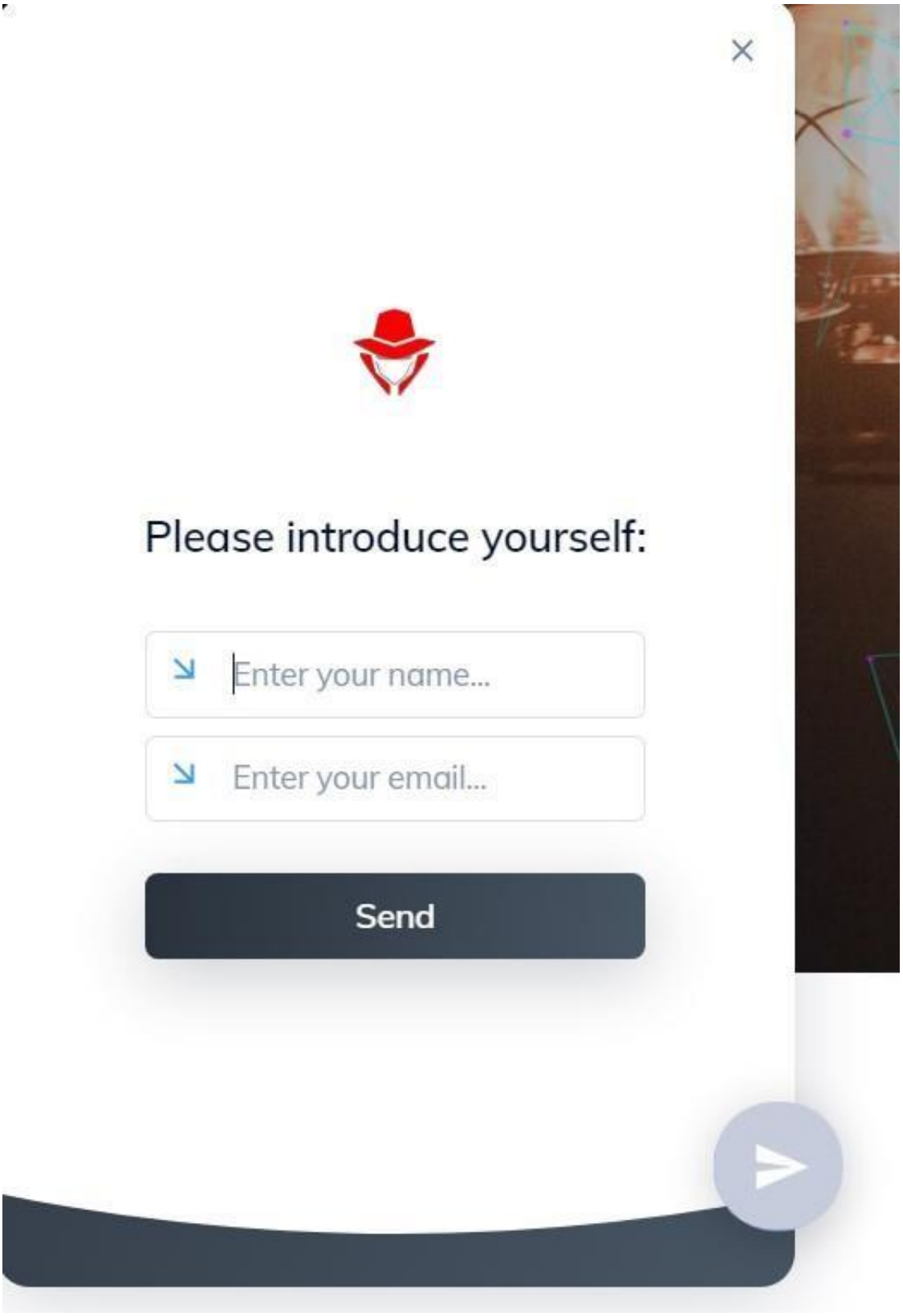
2. User Identity Authentication and Authorization

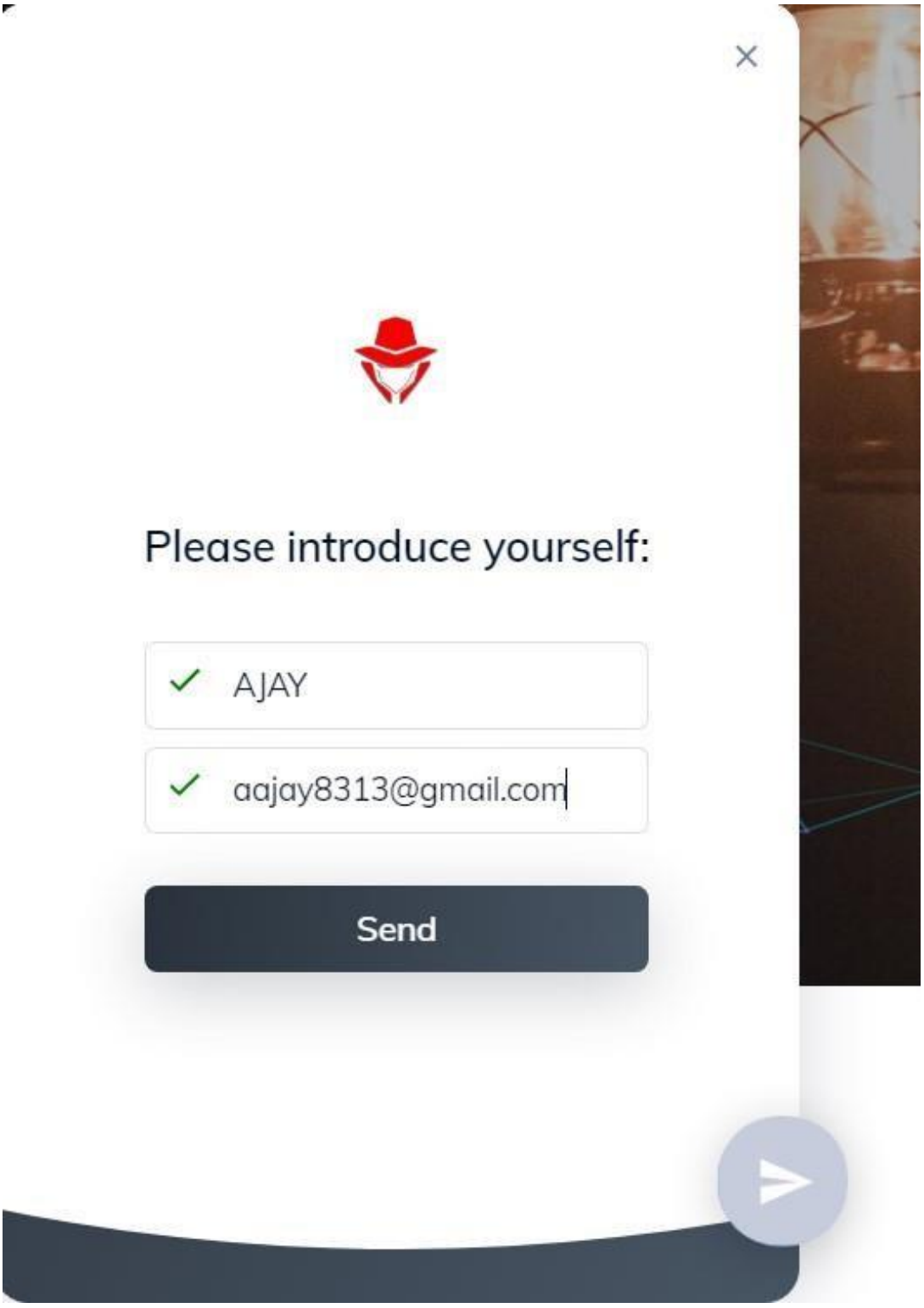
User authentication is used to identify that a user is verified with valid and secure login credentials, such as a username and password. The credentials are exchanged for a secure authentication token; this token is used throughout the user’s session.

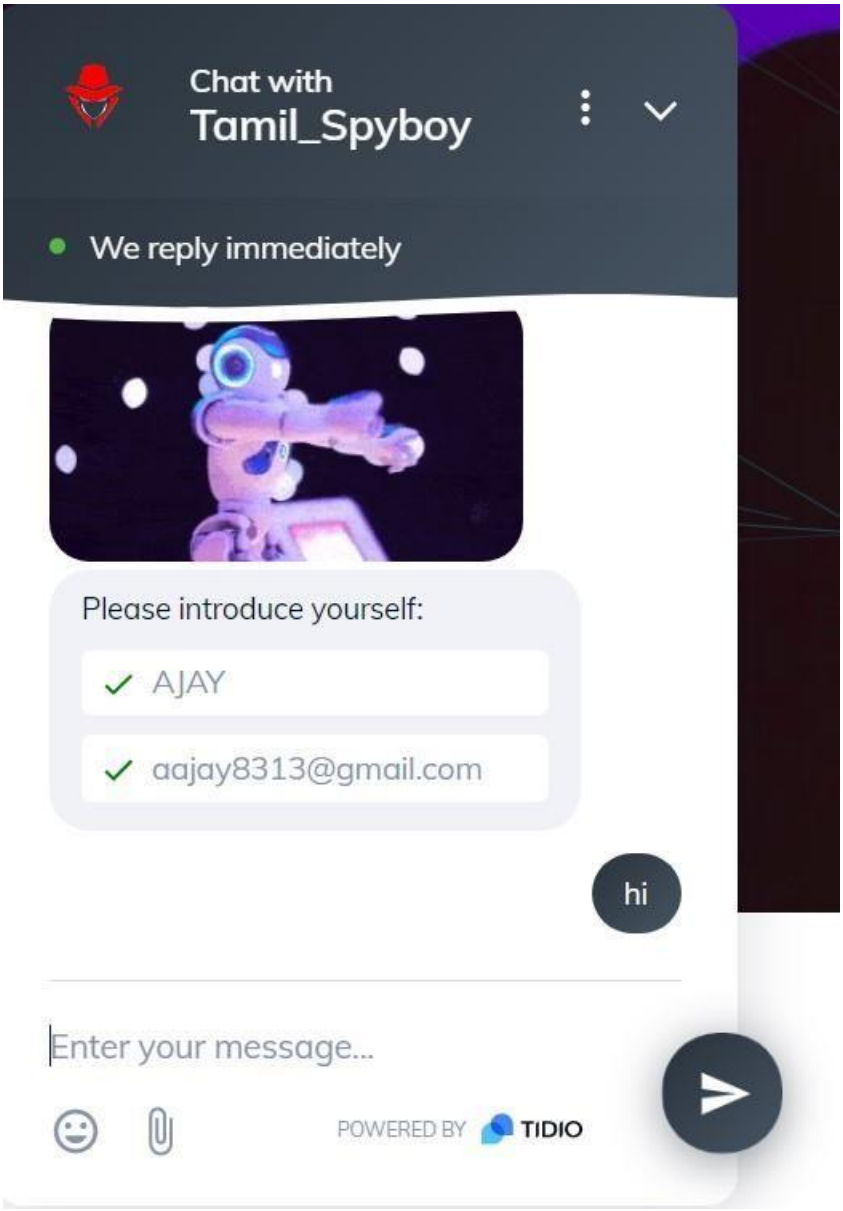
One of the security measures used in chatbots, especially on the banking cases, is authentication timeouts, when the generated token can only be used for a preset amount of time. After that, the system will be forced to make a new one.

Two-factor authentication is another way to verify the identity of a user by asking them to verify their account through an email and text message. This authentication technique also helps with authorization by granting the right person access and keeping the information in the right hands.


Hello
We reply immediately
Hi I'm a bot working for the www.spyberrys.co.in Choose one of the options below or write a message to reach our staff.
Enter your message...
POWERED BY TIDIO


Please introduce yourself:
Enter your name...
Enter your email...
Send


Please introduce yourself:
AJAY
aajay8313@gmail.com
Send



Chapter 6

CONCLUSION&FUTURE WORK

6.1 Conclusions

The research designed and implemented a real-time Honeynet system for detecting and preventing system attacks. System services on Apache Webserver, MYSQL, FTP and SMTP were used to lure attackers. The

experiment was conducted for 10 days. During this period, almost 500,000 attacks were captured. The results of this study shows the top five IP addresses from the world; the top five countries of origin of the attacks, top ten attacks by day, top ten passwords and usernames used by the intruders, and the top ten sources port used for the attacks.

6.2 The research results are useful for addressing one main objective

How should honeypots be made more attractive for hackers to spend more time to provide hacking evidences?

Ans: The honeypots attracted voluminous attacks and evidences of passwords, usernames, and source ports from several countries. However, the File Transfer Protocol server on Honeypot 3 appeared more lucrative for attackers, it attracted the most attacks of 2065 events.

6.3 Future Work

There is more research work that can be conducted to reduce the amount of manual intervention required to add to or modify new honeypots to the honeynet system. In particular, further study can pursue the use of a centralized system management called Ansible to capture and record all activities performed on the honeypots.

REFERENCES

1. Akkaya, D., & Thalgott, F. (2010). Honeypots in network security. Retrieved from http://www.diva-portal.org/smash/get/diva2:327476/fulltext01.

2. Anicas, M. (2015). How to install elasticsearch, logstash, and kibana (ELK Stack) on Ubuntu 14.04. Retrieved from Digital Ocean: https://www.digitalocean.com/community/ tutorials/how-to-install-

3. Autonomous Rule Creation for Intrusion Detection https://inldigitallibrary.inl.gov/sites/sti/sti/5025964.pdf

4. DDoS Attack Detection Method Based on Network Abnormal Behavior in big data environment https://arxiv.org/ftp/arxiv/papers/1903/1903.11844.pdf